

Politechnika Łódzka



IADPŁ

INSTYTUT AUTOMATYKI
POLITECHNIKI ŁÓDZKIEJ

Zakład sterowania robotów

ROBOTY MOBILNE

Instrukcja laboratoryjna 3

Podstawy ROS 2 oraz opis środowiska laboratoryjnego



Spis treści

1	Podstawy ROS 2	3
1.1	Wprowadzenie do ROS 2	3
1.1.1	Dlaczego warto poznać ROS 2?	3
1.1.2	Przykłady zastosowań ROS 2	3
1.2	Węzły w ROS 2	4
1.3	ROS Topic	5
1.3.1	Przykład - Radio	5
1.3.2	Przykład - Powiadomienia w aplikacji pogodowej	5
1.3.3	Podstawowe polecenia podczas pracy z tematami w terminalu	6
1.3.4	Inne cechy ROS Topic	6
1.4	ROS Service	7
1.4.1	Przykład - Włączenie lub wyłączenie światła	7
1.4.2	Przykład - Pobranie temperatury z czujnika	8
1.4.3	Przykład - Odczyt aktualnej prognozy pogody	8
1.4.4	Inne cechy ROS Services	8
1.5	ROS Action	9
1.5.1	Przykład 1 - Pobieranie dużego pliku z internetu	9
1.5.2	Przykład 2 - Robot mobilny jadący do wyznaczonego punktu	10
1.6	Niektóre narzędzia ROS 2	10
1.6.1	Wizualizacja danych - RViz	10
1.6.2	Narzędzia RQT	11
1.6.3	Nagrywanie i odtwarzanie danych - rosbag	13
1.6.4	TF2 – Transformacje układów współrzędnych w ROS 2	13
1.6.5	Gazebo Classic	14
1.6.6	Gazebo	15
2	Opis środowiska laboratoryjnego	16
2.1	Ogólna konfiguracja wszystkich stanowisk	16
2.2	Roboty	17
2.2.1	Opis robota	17
2.2.2	Stanowisko 1 – napęd różnicowy	18
2.2.3	Stanowisko 2 – napęd skid-steer	19
2.2.4	Stanowisko 3 – napęd car-like (Ackermana)	20
2.2.5	Stanowisko 4 – napęd omni (swerve drive)	21
2.2.6	Stanowisko 5 – napęd omni (mecanum)	22
2.3	Konfiguracja adresów IP i ROS_DOMAIN_ID	23
2.4	Komunikacja pomiędzy węzłami ROS2	24
2.4.1	Robot	24

2.4.2	Komputer PC	24
-------	-----------------------	----

1 Podstawy ROS 2

1.1 Wprowadzenie do ROS 2

Wyobraź sobie, że masz możliwość stworzenia własnego robota – takiego, który samodzielnie porusza się po pokoju, unika przeszkód, rozpoznaje przedmioty i reaguje na polecenia. Brzmi jak coś z przyszłości? Dzięki ROS 2 (Robot Operating System 2) – nowoczesnemu frameworkowi do programowania robotów – możesz to zrobić szybciej i łatwiej, niż myślisz!

ROS 2 to zbiór narzędzi (m.in. do archiwizacji i analizy danych) i bibliotek (pakietów), które pomagają w programowaniu robotów. To coś w rodzaju „systemu operacyjnego dla robotów” – pozwala na komunikację między ich elementami, zarządzanie danymi z czujników, sterowanie ruchem i wiele więcej. Dzięki niemu nie musisz pisać wszystkiego od zera – możesz skorzystać z gotowych rozwiązań i skupić się na tym, co najważniejsze: tworzeniu funkcjonalnego i inteligentnego robota.

Możesz myśleć o ROS 2 jak o zestawie klocków LEGO – każdy moduł odpowiada za inne zadanie (np. nawigację, sterowanie silnikami, analizę obrazu), a Twoim zadaniem jest połączenie ich w całość i dostosowanie do swoich potrzeb.

1.1.1 Dlaczego warto poznać ROS 2?

- To przyszłość robotyki – ROS 2 jest używany w robotach mobilnych, dronach, pojazdach autonomicznych i systemach przemysłowych. Jeśli chcesz pracować w tej branży, znajomość ROS 2 to ogromna zaleta.
- Łatwość nauki i gotowe narzędzia – zamiast pisać skomplikowany kod od podstaw, możesz skorzystać z istniejących bibliotek i przykładów. ROS 2 pozwala szybko testować pomysły, bez konieczności budowania fizycznego robota – można pracować w symulacji.
- Modularność – aplikacje w ROS 2 składają się z małych, niezależnych programów (nazywanych węzłami), które komunikują się ze sobą. Dzięki temu łatwo rozszerzać projekt i dodawać nowe funkcje.
- Działa na różnych platformach – ROS 2 obsługuje zarówno komputery PC, jak i Raspberry Pi oraz roboty mobilne. To oznacza, że możesz pisać kod na laptopie, a potem uruchomić go na prawdziwym robocie!
- Licencja open-source – ROS 2 jest dostępny za darmo i rozwijany przez społeczność programistów, badaczy i firm z całego świata.

1.1.2 Przykłady zastosowań ROS 2

- Boston Dynamics
https://www.youtube.com/watch?v=Kf63_hbmUWE&ab_channel=SaketPradhan
https://www.youtube.com/watch?v=yyGrHmtJWfU&ab_channel=LuisCruz
https://www.youtube.com/watch?v=mqDncPrTI2w&ab_channel=VentureX-FutureTech
- mini robot o kinematyce typu Ackermann
https://www.youtube.com/watch?v=22sJoi4Kg0&ab_channel=YahboomTechnology
- Symulacja - automatyczne dokowanie do stacji ładowania
https://www.youtube.com/watch?v=TY-FOH81yk4&ab_channel=AutomaticAddison

1.2 Węzły w ROS 2

W systemie **ROS 2** podstawowym elementem struktury programu jest **węzeł** (ang. *node*). Każdy węzeł stanowi niezależny proces (program), który realizuje określoną funkcję w ramach systemu robotycznego. Może on działać samodzielnie lub współpracować z innymi węzłami w celu wymiany danych i koordynacji zadań. Dzięki takiej architekturze system jest modułowy — poszczególne elementy można uruchamiać, zatrzymywać, testować i modyfikować niezależnie od siebie.

Węzeł w ROS 2 może pełnić różne role, takie jak:

- przetwarzanie danych z czujników,
- realizacja algorytmu sterowania,
- planowanie ruchu,
- wizualizacja informacji lub komunikacja z interfejsem użytkownika.

Węzły komunikują się ze sobą przy użyciu czterech podstawowych mechanizmów:

- **Tematy** (ang. *topics*) — służą do wymiany danych w modelu publikacja-subskrypcja. Jeden węzeł może publikować informacje, a inne mogą je subskrybować, odbierając aktualne dane w czasie rzeczywistym.
- **Serwisy** (ang. *services*) — umożliwiają komunikację w modelu żądanie-odpowiedź. Węzeł może wysłać zapytanie do innego węzła i poczekać na wynik operacji, np. prośbę o wykonanie pomiaru lub zmianę stanu robota.
- **Akcje** (ang. *actions*) — wykorzystywane są w sytuacjach, gdy zadanie trwa dłużej i może być w trakcie przerywane lub monitorowane. Przykładem może być ruch robota do określonego punktu, podczas którego system przekazuje informacje o postępie i pozwala na anulowanie zadania.
- **Parametry** — stanowią zestaw wartości konfiguracyjnych pozwalających na zmianę zachowania węzła bez modyfikacji jego kodu, np. częstotliwości publikacji czy współczynników regulatora.

Uruchomienie pojedynczego węzła w systemie ROS 2 odbywa się za pomocą polecenia:

```
1 ros2 run <nazwa\_paczki> <nazwa\_węzła>
```

Polecenie to inicjuje działanie jednego węzła należącego do określonej paczki.

W sytuacji, gdy konieczne jest jednoczesne uruchomienie kilku węzłów (np. czujników, kontrolera i interfejsu wizualizacji), wykorzystuje się pliki uruchomieniowe `.launch.py`. Pozwalają one na definiowanie zestawów węzłów, ich parametrów i kolejności startu w jednym miejscu, co ułatwia zarządzanie bardziej złożonymi systemami.

Uruchomienie takiego pliku realizowane jest poleceniem:

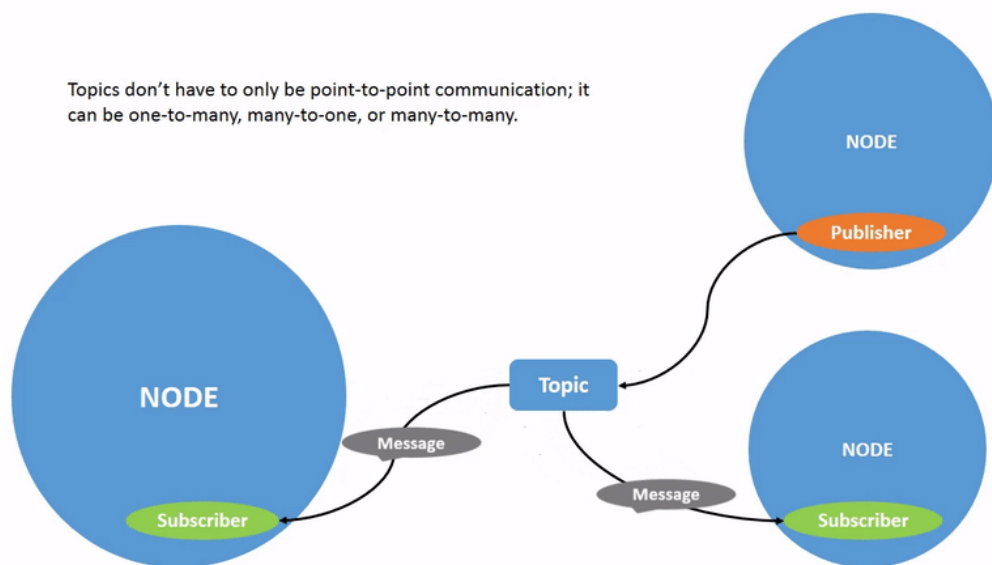
```
1 ros2 launch <nazwa\_paczki> <nazwa\_pliku.launch.py>
```

Dzięki temu jednym poleceniem można rozpocząć działanie całego środowiska robota, obejmującego zarówno węzły sterujące, jak i pomocnicze moduły komunikacyjne, wizualizacyjne czy diagnostyczne. Architektura oparta na węzłach sprawia, że ROS 2 jest środowiskiem elastycznym i skalowalnym. Węzły mogą działać na jednym komputerze lub być rozproszone pomiędzy wieloma urządzeniami połączonymi w sieć, zachowując spójność komunikacji i wymiany danych.

1.3 ROS Topic

ROS Topic jest jednym z podstawowych sposobów wymiany informacji między węzłami (nodes). Jest to jednokierunkowa i rozgłoszeniowa forma komunikacji, co oznacza, że jeden węzeł może wysyłać dane, a inne mogą je odbierać bez konieczności nawiązywania bezpośredniego połączenia.

Komunikacja w ROS 2 za pomocą Topiców opiera się na modelu **Publisher-Subscriber**.



Rys. 1.1: Komunikacja dla ROS Topic. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>

ROS Topic składa się z:

- **Publisher** – Wysyła wiadomości na określony temat (topic). Może to być np. stacja radiowa nadająca sygnał.
- **Subscriber** – Odbiera wiadomości publikowane na danym temacie. Może to być radioodbiornik, który dostraja się do danej częstotliwości i odbiera transmisję.
- **Topic/Temat** (kanał komunikacji) – Określa nazwę i typ przesyłanych wiadomości, np. jak częstotliwość radiowa lub grupa na WhatsAppie.

1.3.1 Przykład - Radio

W tym modelu każdy, kto dostroi się do odpowiedniej częstotliwości (topic), otrzyma te same dane.

- **Topic** (temat) - 103.5 FM
- **Publisher** - Stacja radiowa nadająca wiadomości
- **Subscriber** - Każda osoba, która włączy radio na tej częstotliwości
- **Wiadomość** - Muzyka lub komunikaty informacyjne

1.3.2 Przykład - Powiadomienia w aplikacji pogodowej

Każdy, kto ma subskrybowaną daną lokalizację, otrzymuje te same dane.

- **Topic** (temat) - Aktualna pogoda w danym mieście
- **Publisher** - Serwis pogodowy aktualizujący dane co godzinę
- **Subscriber** - Użytkownicy aplikacji pogodowej, którzy otrzymują powiadomienia o deszczu
- Wiadomość - „Za godzinę zacznie padać deszcz”

1.3.3 Podstawowe polecenia podczas pracy z tematami w terminalu

Jednym z najczęściej używanych poleceń jest:

```
1  ros2 topic echo <nazwa\_tematu>
```

Polecenie to pozwala na wyświetlenie w terminalu danych publikowanych w określonym temacie w czasie rzeczywistym. Dzięki temu można łatwo sprawdzić, jakie informacje przesyła dany węzeł — na przykład odczyty z czujników, prędkość robota lub stan baterii.

Drugie ważne polecenie to:

```
1  ros2 topic pub <nazwa\_tematu> <typ\_wiadomości> <wartości>
```

Umożliwia ono ręczne publikowanie danych do wybranego tematu bez konieczności uruchamiania dedykowanego węzła. Może być użyte, na przykład, do przetestowania reakcji robota na określone polecenia sterujące lub do wysłania pojedynczej wiadomości w celu sprawdzenia poprawności komunikacji.

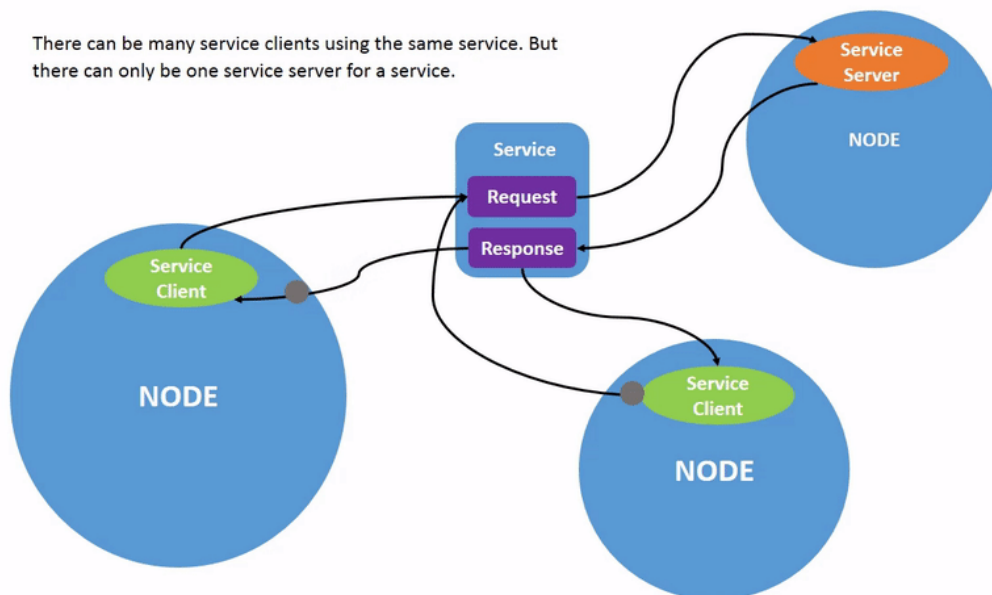
1.3.4 Inne cechy ROS Topic

- **Asynchroniczność** – Publisher nie musi czekać na odpowiedź, wysyła dane niezależnie od tego, czy ktoś je odbiera.
- **Wielu subskrybentów** – Jeden Publisher może wysyłać dane do wielu subskrybentów jednocześnie.
- **Brak bezpośredniego połączenia** – Publisher nie „wie”, kto go słucha, wysyła dane na określony temat i każdy zainteresowany może je odebrać.
- **Luźne powiązanie** – Subskrybenci mogą pojawiać się i znikać, a Publisher nadal działa bez problemu.

1.4 ROS Service

ROS Services jest kolejnym sposobem wymiany informacji między węzłami. Jest to mechanizm umożliwiający jednorazową wymianę informacji między klientem a serwerem.

Komunikacja w ROS2 za pomocą Serwisów opiera się na modelu **Request-Response (żądanie-odpowiedź)**. Oznacza to, że klient wysyłający żądanie z jednego węzła oczekuje na uzyskanie odpowiedzi od serwera umieszczonego w innym węźle.



Rys. 1.2: Komunikacja dla serwisów. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>

Elementy składowe ROS2 Services:

- Serwer serwisu (service server) – udostępnia określoną usługę. Nasłuchuje na pojawienie się żądania i odpowiada na nie.
- Klient serwisu (service client) – wysyła żądanie do serwera i czeka na odpowiedź.
- Temat serwisu (service names) - unikalna nazwa kanału komunikacyjnego umożliwiającą klientowi wysłanie wiadomości do serwera. Na temacie klient wysyła typ wiadomości zgodny z typem obsługiwanym przez serwer.
- Żądanie (Request) - wiadomość wysyłana przez klienta
- Odpowiedź (Response) – wiadomość zwracana przez serwer.

1.4.1 Przykład - Włączenie lub wyłączenie światła

Jednym z przykładów ilustrujących zrozumienie serwisów jest np. sterownik systemu inteligentnego oświetlenia udostępniający usługę umożliwiającą zmianę stanu światła. Aplikacja sterująca inteligentnym domem wysyła żądanie zmiany stanu światła. Sterownik systemu oświetlenia odbiera żądanie i wykonuje operację.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - Aplikacja sterująca inteligentnym domem
- Serwer serwisu - Sterownik systemu

- Żądanie - Wiadomość wysyłana przez klienta zawierająca informacje o tym, czy światło ma zostać włączone czy wyłączone.
- Odpowiedź - wysyłana jest przez serwer i informuje o tym czy światło zostało włączone lub wyłączone.

1.4.2 Przykład - Pobranie temperatury z czujnika

System zarządzania klimatem udostępnia usługę pozwalającą na pobranie aktualnej temperatury otoczenia. System kontroli wentylacji pyta czujnik o aktualną temperaturę, aby dostosować działanie klimatyzacji. Czujnik temperatury odpowiada natychmiast bieżącą wartością pomiaru.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - System kontroli wentylacji
- Serwer serwisu - Czujnik temperatury
- Żądanie - Wiadomość wysyłana przez klienta z prośbą o podanie bieżącej temperatury.
- Odpowiedź - Wiadomość zwracana przez serwer zawierająca aktualny odczyt temperatury w stopniach Celsjusza.

1.4.3 Przykład - Odczyt aktualnej prognozy pogody

System pogodowy udostępnia usługę pozwalającą na pobranie bieżącej prognozy pogody dla wybranej lokalizacji. Aplikacja pogodowa wysyła zapytanie na serwer o prognozę pogody. Centralny serwer meteorologiczny odbiera zapytanie, pobiera dane z czujników pogodowych i zwraca prognozę.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - Aplikacja pogodowa
- Serwer serwisu - Centralny serwer meteorologiczny
- Żądanie - Wiadomość wysyłana przez klienta z prośbą o podanie prognozy pogody.
- Odpowiedź - Wiadomość zwracana przez serwer zawierająca aktualny odczyt prognozy pogody.

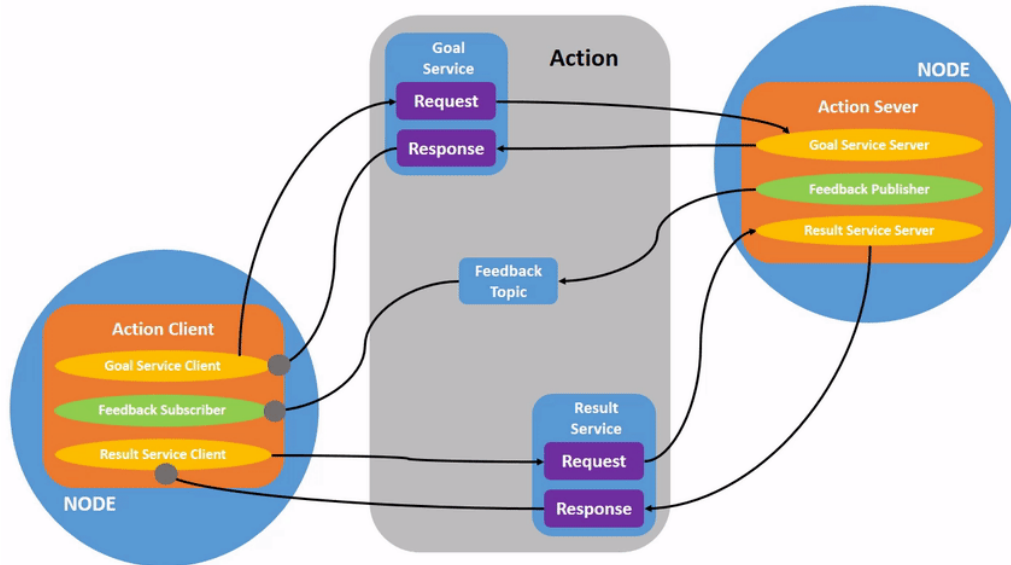
1.4.4 Inne cechy ROS Services

- Wymagają natychmiastowej odpowiedzi.
- Mają jednorazowy charakter (po zakończeniu operacji nie trzeba monitorować dalszego przebiegu).

Serwisy nie są odpowiednie dla działań długotrwałych, które wymagają aktualizacji statusu (np. jazda robota do celu, skanowanie obszaru), ponieważ do takich zadań lepiej nadają się akcje (actions).

1.5 ROS Action

ROS2 Action (Akcje) są kolejnym sposobem wymiany informacji między węzłami. Akcje są stosowane, gdy wykonywane zadanie zajmuje więcej czasu i wymaga monitorowania postępu lub możliwości anulowania wysłanej akcji.



Rys. 1.3: Komunikacja dla akcji. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

Akcja składa się z trzech głównych elementów:

- **Goal** (cel) – Klient wysyła żądanie wykonania zadania do serwera akcji.
- **Feedback** (informacja zwrotna) – Serwer akcji może okresowo wysyłać klientowi informacje o postępie.
- **Result** (wynik) – Po zakończeniu zadania serwer zwraca końcowy rezultat.

Komunikacja opiera się na wzorcu **Klient-Serwer**:

- Klient wysyła żądanie (goal) do Serwera Akcji.
- Serwer rozpoczyna realizację zadania i może wysyłać feedback.
- Po zakończeniu zadania serwer wysyła końcowy wynik.

Dodatkowo, klient może w każdej chwili anulować zadanie, jeśli np. nie jest już potrzebne.

1.5.1 Przykład 1 - Pobieranie dużego pliku z internetu

Jednym z przykładów umożliwiającym zrozumienie działania akcji jest pobieranie dużych plików może zajmować wiele godzin. Z tego powodu użytkownik końcowy chce monitorować postęp pobierania pliku.

- **Goal** (Cel) - Kliknięcie przycisku „Pobierz” na stronie internetowej wywołuje akcję pobierania pliku.
- **Feedback** (Informacja zwrotna) - Przeglądarka pokazuje pasek postępu: „10% pobrane”, „50% pobrane”, „80% pobrane”.
- **Result** (Wynik) - Plik zostaje pobrany i zapisany na dysku.

Użytkownik może anulować pobieranie pliku, jeśli zobaczy, że plik jest za duży lub pobiera niewłaściwy plik.

1.5.2 Przykład 2 - Robot mobilny jadący do wyznaczonego punktu

- **Goal** (Cel) - Robot otrzymuje polecenie, aby pojechać do określonego punktu (np. współrzędne X, Y w przestrzeni).
- **Feedback** (Informacja zwrotna) - Robot raportuje działania:
 - Planowanie trasy do punktu (X, Y)
 - Rozpoczęcie jazdy
 - Przejechano 10% trasy
 - Przejechano 50% trasy
 - Wykryto przeszkodę. Nastąpiło wygenerowanie nowej trasy
- **Result** (Wynik) - Robot osiąga docelową pozycję i informuje o zakończeniu zadania.

Można przerwać zadanie w dowolnym momencie, np. zmieni się cel misji.

1.6 Niektóre narzędzia ROS 2

1.6.1 Wizualizacja danych - RViz

RViz (ROS Visualization) to narzędzie do wizualizacji danych w ROS 2. Umożliwia wyświetlanie informacji z różnych czujników, analizę transformacji układów współrzędnych (**tf**), wizualizację map, trajektorii oraz modeli robotów. Narzędzie wykorzystywane jest m.in. do testowania algorytmów percepcji, nawigacji i sterowania.

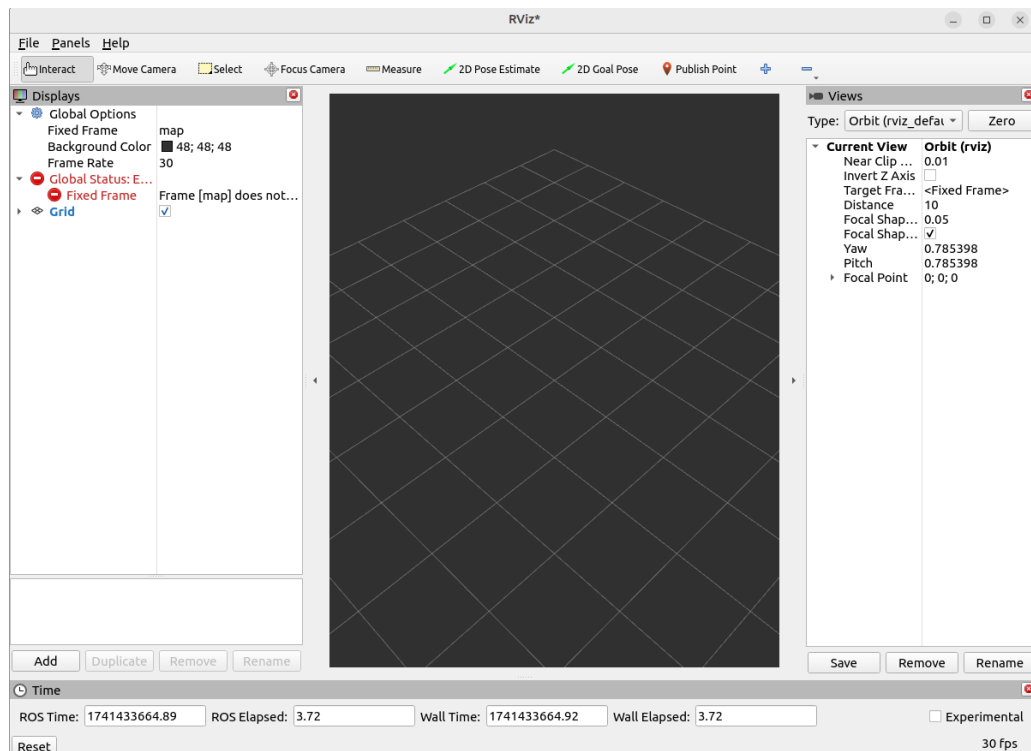
Podstawowe funkcjonalności RViz:

- Wyświetlanie chmur punktów – np. dane z LIDAR-a w formacie `sensor_msgs/PointCloud2`.
- Podgląd obrazów z kamer – np. z tematu `/camera/image_raw`.
- Wizualizacja układów współrzędnych. Wyświetlanie ramek **tf** w celu weryfikacji poprawności działania transformacji.
- Obsługa map i trajektorii (np. wizualizacja `nav_msgs/Path`). Znajduje zastosowanie podczas testów algorytmów SLAM i nawigacji.
- Podgląd położenia robota i innych obiektów - dane w formacie `geometry_msgs/PoseStamped`

Uruchamianie RViz:

```
1  ros2 run rviz2 rviz2
```

Następnie pojawia się okno:



Rys. 1.4: Okno RViz

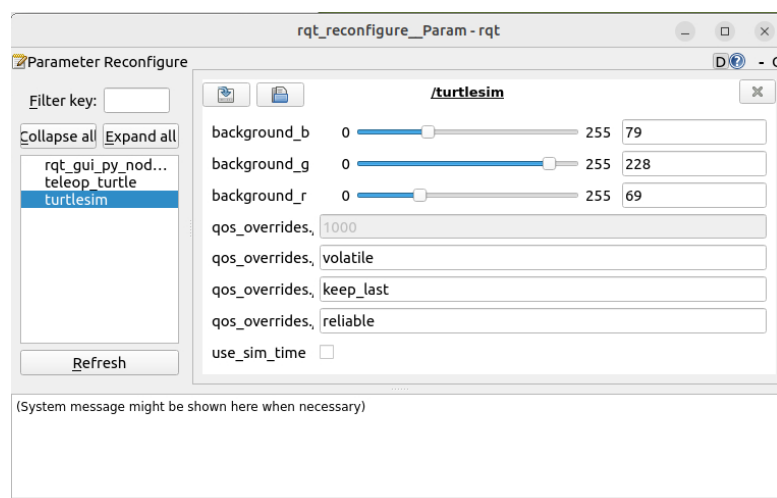
RViz uruchomi się z domyślną konfiguracją. Można także załadować własny plik konfiguracyjny:

```
1 ros2 run rviz2 rviz2 -d my_config.rviz
```

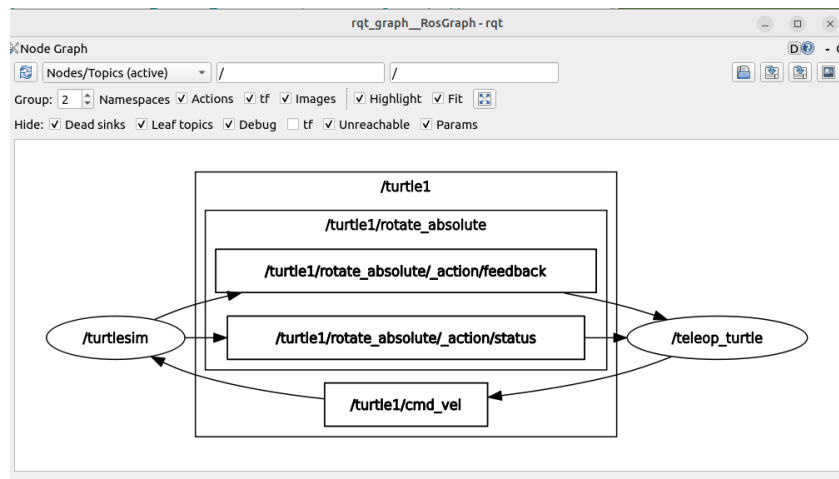
1.6.2 Narzędzia RQT

RQT to zbiór aplikacji graficznych opartych na Qt, które umożliwiają wizualizację, monitorowanie i konfigurację ROS 2 w sposób interaktywny. Dzięki narzędziom RQT można w łatwy sposób analizować strukturę komunikacji między węzłami, podglądać wiadomości przesyłane w tematach, a także modyfikować parametry działających komponentów systemu.

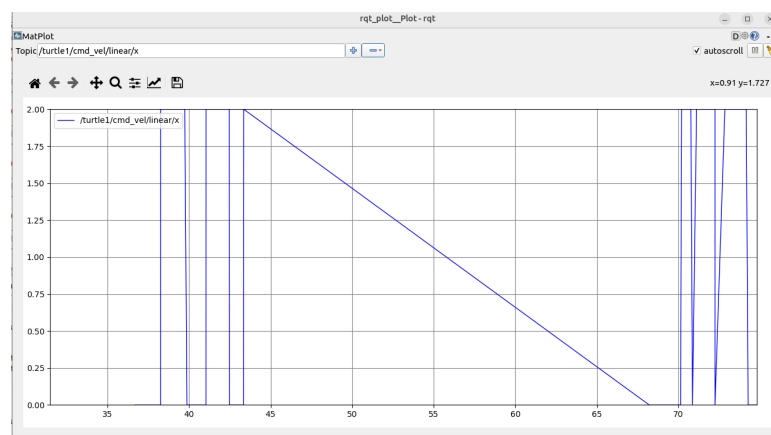
Przykłady niektórych narzędzi



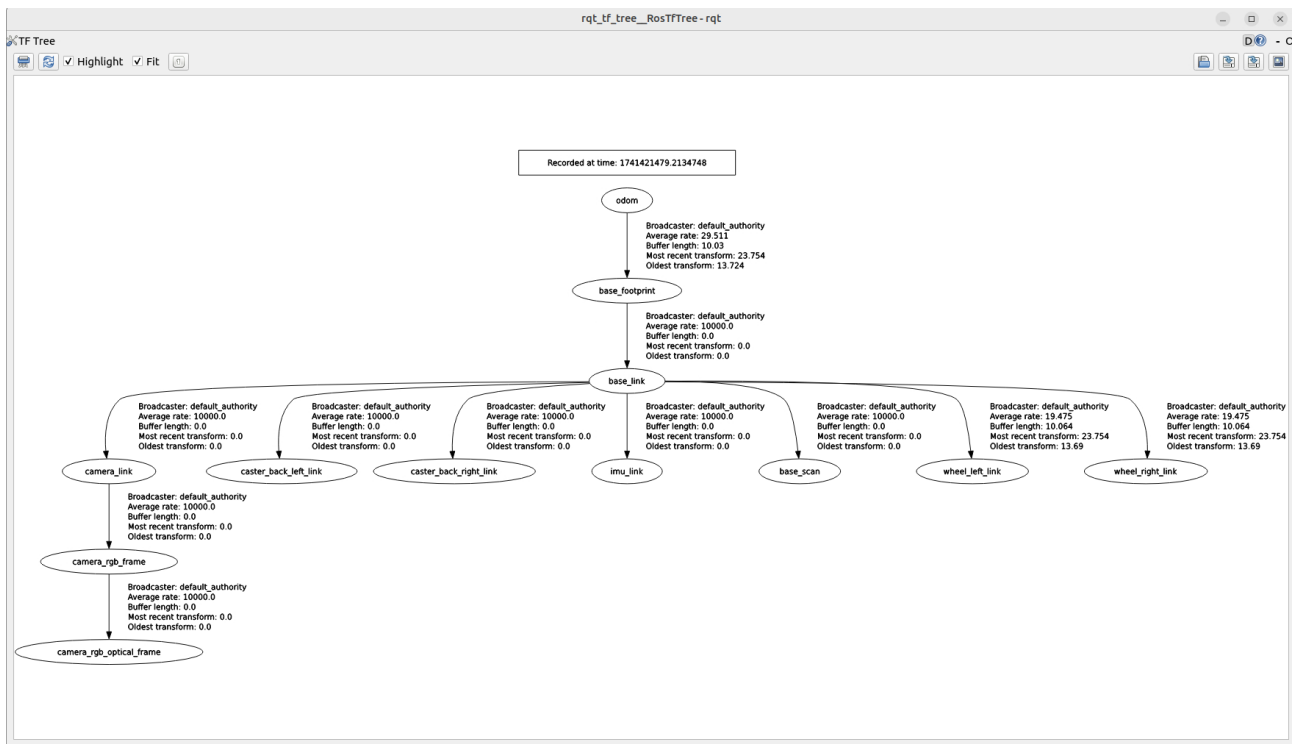
Rys. 1.5: GUI dla narzędzia RQT_RECONFIGURE - możliwość dynamicznej zmiany parametrów



Rys. 1.6: GUI dla narzędzia RQT_GRAPH - wyświetlanie zależności i połączeń między różnymi węzłami



Rys. 1.7: GUI dla narzędzia RQT_PLOT - rysowanie wykresów na podstawie odbieranych danych



Rys. 1.8: GUI dla narzędzia RQT_TF_TREE - wizualizacja połączeń opublikowanych układów współrzędnych TF

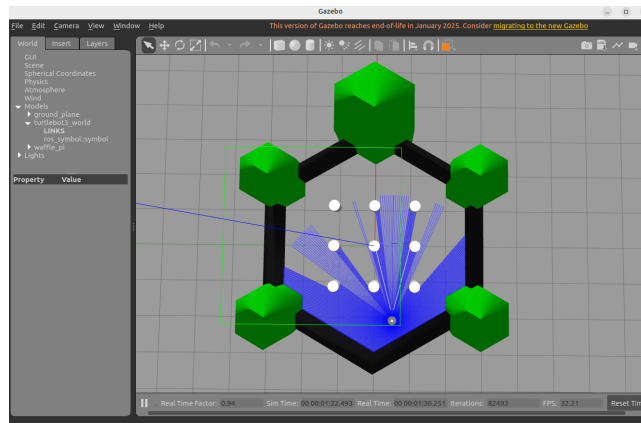
- Współpraca z RViz – wizualizacja układów współrzędnych w czasie rzeczywistym.

Zastosowanie TF2 w ROS2:

- Roboty mobilne – śledzenie pozycji platformy względem mapy.
- Manipulatory – określanie pozycji chwytaka względem podstawy robota.
- Systemy wizyjne – analiza położenia kamer i obiektów w otoczeniu robota.
- Fuzja danych sensorycznych – przekształcanie danych z różnych czujników do wspólnego układu odniesienia.

1.6.5 Gazebo Classic

Gazebo Classic to zaawansowany symulator do testowania i rozwijania systemów robotycznych w ROS 2. Pozwala na realistyczne odwzorowanie fizyki, interakcji robotów ze środowiskiem oraz pracy czujników w wirtualnym świecie. Jest szeroko stosowany w badaniach, edukacji oraz przemyśle do testowania algorytmów przed wdrożeniem ich na rzeczywiste roboty.



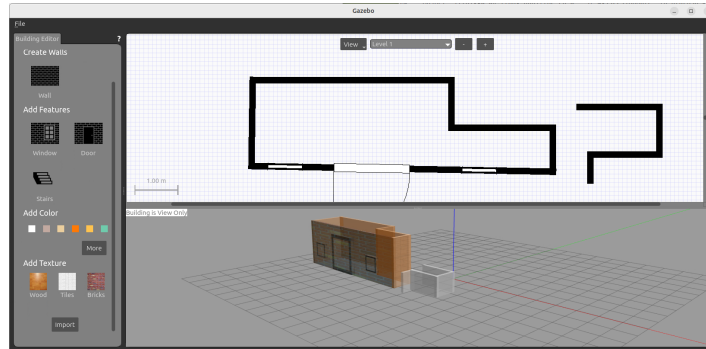
Rys. 1.10: Widok symulowanego robota TurtleBot3 waffle_pi w symulacji

Funkcjonalności Gazebo:

- Zaawansowana fizyka. Gazebo Classic obsługuje różne silniki fizyczne (ODE, Bullet), które symulują:
 - Grawitację i tarcie
 - Zderzenia między obiektami
 - Dynamiczne interakcje robotów z otoczeniem
- Obsługa czujników m.in.:
 - LIDAR-ów
 - Kamer RGB i głębi
 - IMU
 - GPS
- Modelowanie robotów z URDF/XACRO
- Możliwość symulacji wielu robotów
- Integracja z ROS 2. Możliwość uruchamiania symulacji razem z ROS 2 i publikowania danych na tematy ROS 2. Symulowane czujniki mogą być analizowane jak rzeczywiste urządzenia.

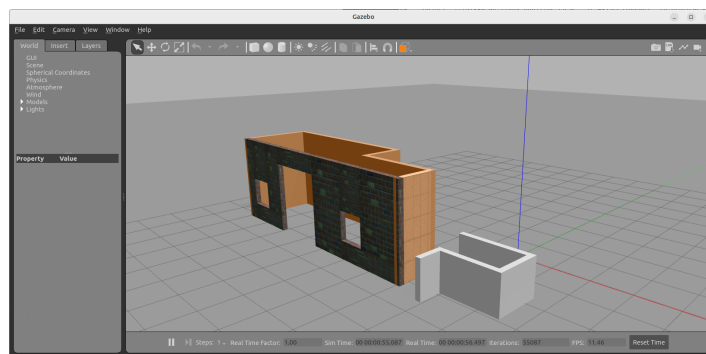
- Dynamiczne modyfikowanie środowiska. Można dodawać obiekty w trakcie symulacji i zmieniać parametry robotów w czasie rzeczywistym.

Edit/Building Editor umożliwia rysowanie wielopoziomowych budynków w środowisku symulacyjnym poprzez dodawanie konfiguracji ścian, drzwi, okien i pięter. Po narysowaniu modelu można go zapisać. Niestety po jego zapisaniu nie będzie można go później edytować. Przykładowy widok z okna przedstawia:



Rys. 1.11: Widok okna Edit/Building Editor

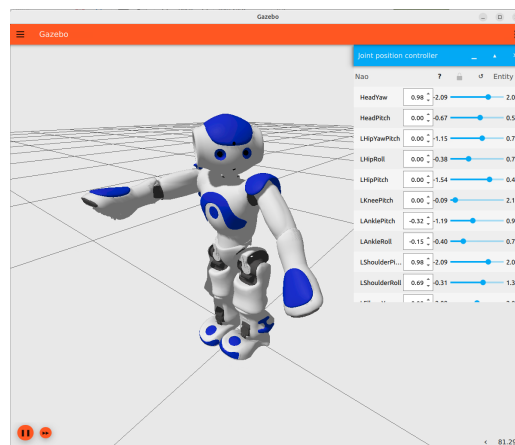
Po zapisaniu modelu świat symulacji wygląda następująco:



Rys. 1.12: Widok świata po zamknięciu edytora do rysowania budynków

1.6.6 Gazebo

Gazebo Ignition (obecnie znane jako Gazebo) to nowa wersja popularnego symulatora Gazebo Classic, zaprojektowana z myślą o bardziej modułowej architekturze, lepszej integracji z ROS 2 oraz poprawionej wydajności.

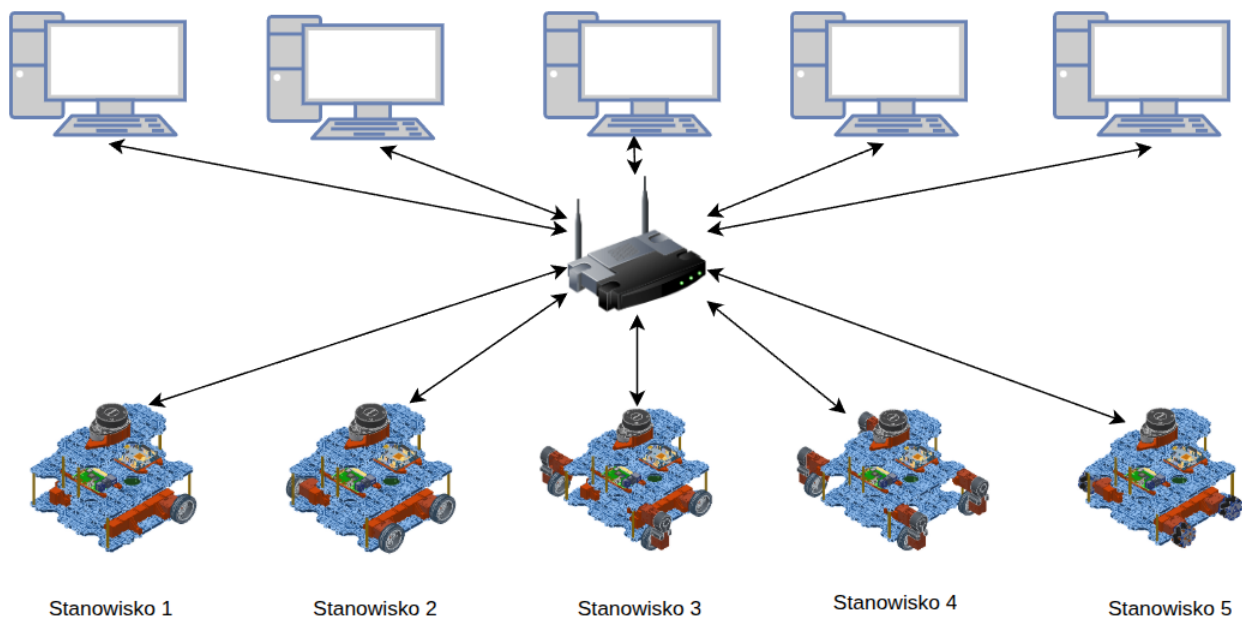


Rys. 1.13: Widok symulowanego robota Nao w symulacji

2 Opis środowiska laboratoryjnego

2.1 Ogólna konfiguracja wszystkich stanowisk

Stanowisko laboratoryjne składa się ze stacjonarnego komputera PC oraz robota. Wszystkie stanowiska i roboty są połączone w jedną sieć. Oznacza to, że z dowolnego komputera w laboratorium można połączyć się z robotem przypisanym do innego stanowiska. Z tego powodu należy zwrócić szczególną uwagę na sposób uruchamiania środowiska oraz ustawienie zmiennej środowiskowej ROS 2 `ROS_DOMAIN_ID`.



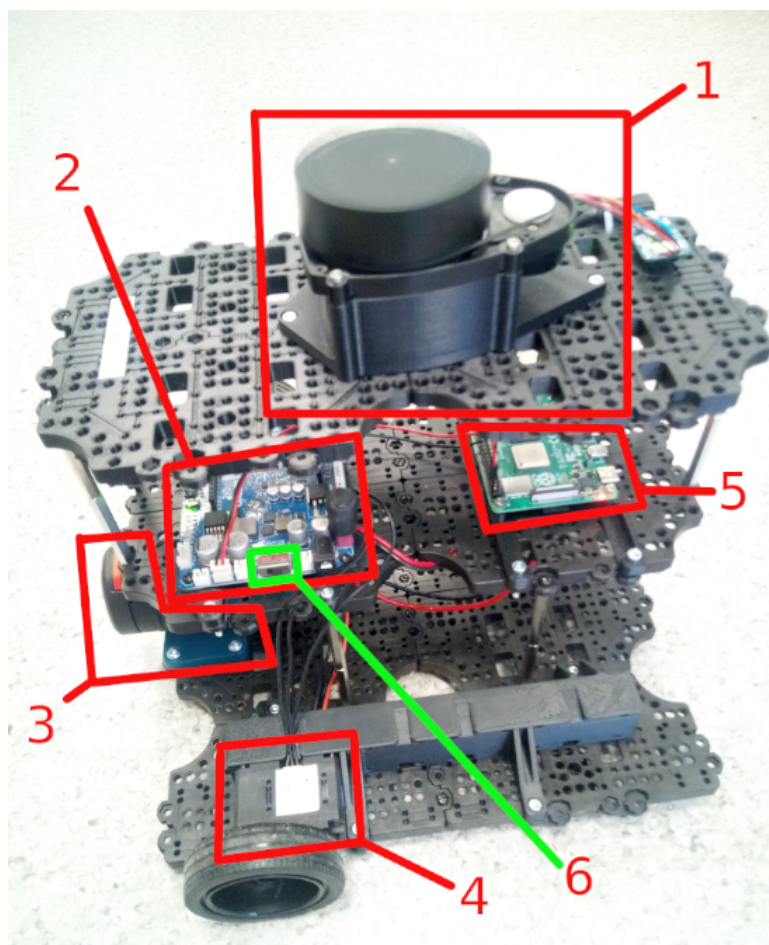
Rys. 2.1: Konfiguracja stanowisk w laboratorium.

Komputery połączone są z routerem za pomocą przewodów, natomiast roboty komunikują się poprzez sieć Wi-Fi. Dla każdego numeru stanowiska przypisana jest odpowiednia kinematyka robota:

1. Stanowisko 1 – napęd różnicowy (TB3_1 RPI4 – 2 wheel diff),
2. Stanowisko 2 – napęd skid-steer (TB3_1 RPI4 – skid-steer 4 wheels),
3. Stanowisko 3 – napęd typu car-like (TB3_1 RPI4 – car-like 4 wheels),
4. Stanowisko 4 – napęd omni (swerve drive) (TB3_1 RPI4 – swerve drive 4 wheels),
5. Stanowisko 5 – napęd omni (mecanum) (TB3_1 RPI4 – mecanum 4 wheels).

2.2 Roboty

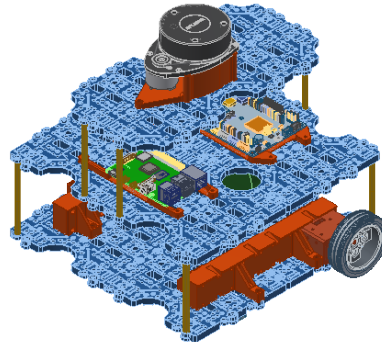
2.2.1 Opis robota



Rys. 2.2: Rzeczywisty robot - najważniejsze elementy.

1. skaner laserowy
2. OpenCR - niskopoziomowy sterownik robota odpowiedzialny za sterowanie napędami oraz odczyt aktualnej prędkości i kąta skrętu z napędów. Połączony jest z **Raspberry Pi 4** i napędami.
3. bateria
4. serwo - napęd robota
5. Raspberry Pi 4 wysokopoziomowy sterownik robota, który łączy się bezprzewodowo z siecią Wi-Fi w laboratorium. Na nim uruchamiany jest węzeł umożliwiający:
 - odbieranie prędkości sterujących napędami z PC i przekazanie ich na niższy poziom aż do OpenCR,
 - odbierania aktualnych informacji o prędkościach i kącie skrętu napędów i opublikowania ich w ROS tak, aby były widoczne na PC.
6. Przełącznik ON/OFF na robocie.

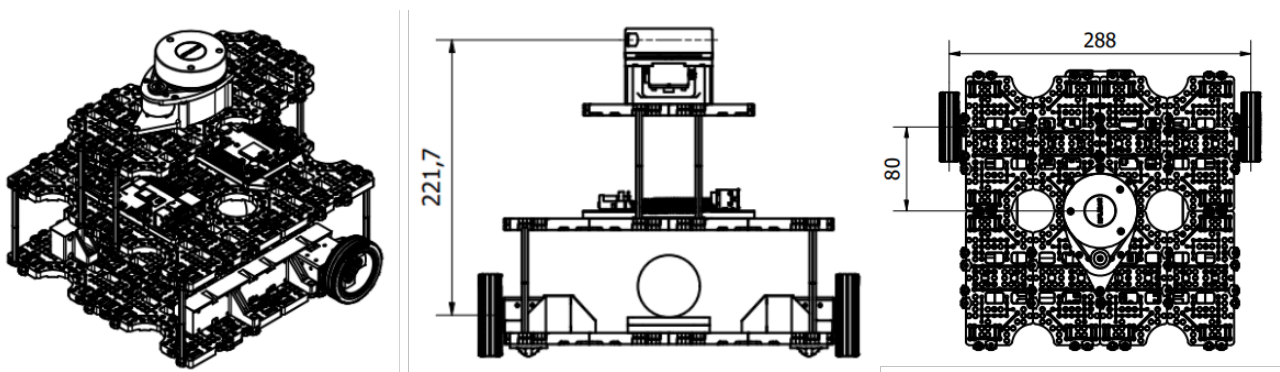
2.2.2 Stanowisko 1 – napęd różnicowy



Rys. 2.3: Rzut izometryczny robota mobilnego z napędem różnicowym.

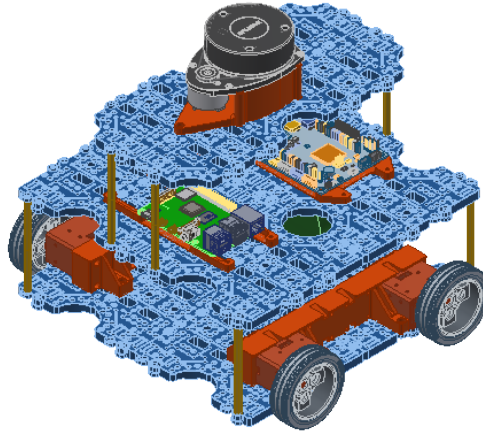
Sterowanie platformą różnicową opiera się na niezależnym napędzaniu dwóch kół po obu stronach robota, co umożliwia precyzyjne manewrowanie poprzez różnicowanie prędkości tych kół. Aby platforma poruszała się na wprost, oba koła muszą obracać się z taką samą prędkością i w tym samym kierunku. Z kolei różnica w prędkościach kół powoduje skręt — im większa różnica, tym ostrzejszy zakręt; jeśli jedno koło obraca się do przodu, a drugie do tyłu, platforma wykonuje pełny obrót wokół własnej osi.

Sterowanie tego typu platformą można realizować poprzez prostą regulację prędkości obrotu każdego koła, co czyni ją łatwą do implementacji w systemach autonomicznych i zdalnie sterowanych. W praktyce sterowanie różnicowe jest szczególnie skuteczne w aplikacjach wymagających dużej manewrowości, takich jak poruszanie się w ograniczonych przestrzeniach. Algorytmy sterujące mogą dynamicznie dostosowywać prędkości kół na podstawie sygnałów z czujników, aby precyzyjnie kontrolować kierunek i trajektorię ruchu robota, umożliwiając omijanie przeszkód i dokładne nawigowanie w wyznaczonym obszarze.



Rys. 2.4: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.2.3 Stanowisko 2 – napęd skid-steer

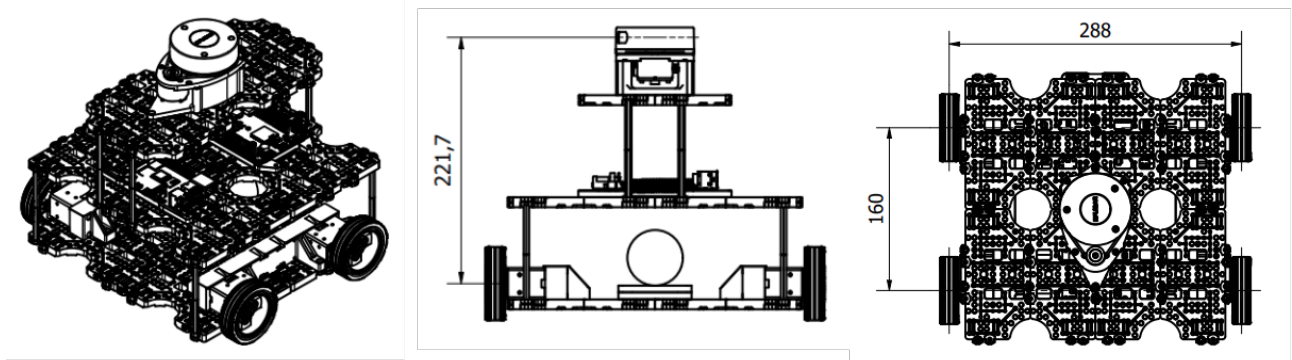


Rys. 2.5: Rzut izometryczny robota mobilnego z napędem różnicowym typu skid-steer.

Napęd skid-steer, stosowany w pojazdach takich jak ładowarki czy roboty terenowe, opiera się na napędzaniu kół (lub gąsienic) po obu stronach pojazdu. Sterowanie polega na różnicowaniu prędkości między lewą a prawą stroną — gdy koła po jednej stronie obracają się szybciej, pojazd skręca w stronę wolniejszej strony, a przy obrocie kół w przeciwnych kierunkach wykonuje obrót w miejscu.

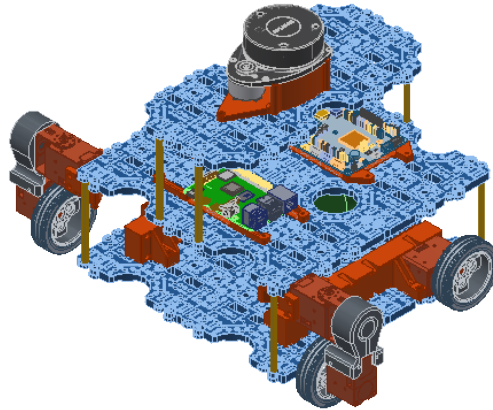
W przeciwieństwie do klasycznego napędu różnicowego, w napędzie skid-steer wszystkie koła są sztywno zamocowane i ustawione równolegle, co oznacza, że skręt wymaga wymuszenia poślizgu kół. Ten poślizg jest niezbędny dla skrętu, lecz wprowadza istotne ograniczenie – zasada braku poślizgu nie jest tu możliwa do spełnienia. W wyniku poślizgu odczyty odometryczne zawierają błędy, ponieważ przemieszczenie platformy nie odpowiada dokładnie obrotom kół, zwłaszcza podczas skręcania. Błędy odometrii mogą powodować nieprecyzyjne odwzorowanie zadanej trasy przez robota, jeśli nie zostaną skorygowane.

Aby zminimalizować te błędy, stosuje się dodatkowe czujniki, takie jak IMU, GPS czy LiDAR, które kompensują odchylenia wynikające z poślizgu i poprawiają dokładność nawigacji. Napęd skid-steer jest odporny i dobrze sprawdza się na trudnym terenie, gdzie poślizg jest naturalny, jednak jego precyzja odometrii na równym podłożu jest niższa niż w przypadku klasycznego napędu różnicowego.



Rys. 2.6: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.2.4 Stanowisko 3 – napęd car-like (Ackermana)

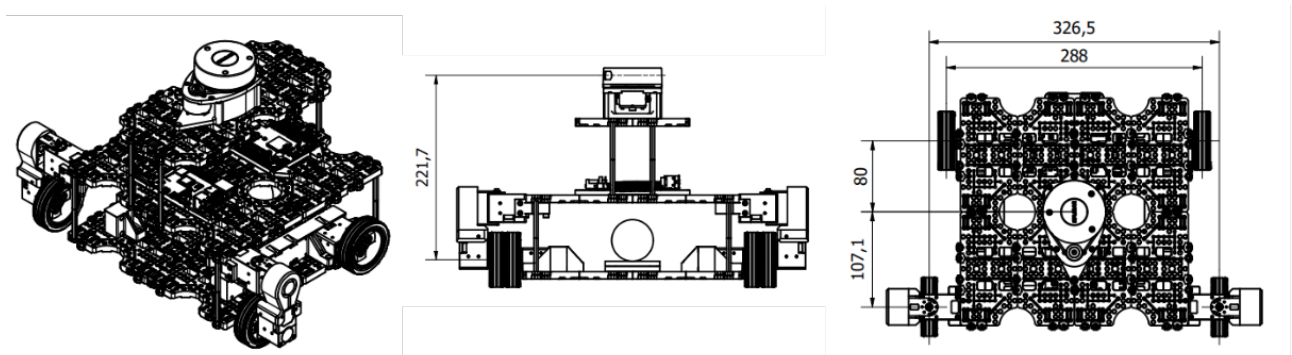


Rys. 2.7: Rzut izometryczny robota mobilnego z napędem Ackermana.

Napęd typu *car-like*, znany również jako napęd Ackermana, jest stosowany w pojazdach takich jak samochody czy niektóre roboty mobilne. Charakteryzuje się tym, że skręcanie odbywa się przez zmianę kąta obrotu przednich kół, podczas gdy tylne koła pozostają skierowane na wprost. W odróżnieniu od napędów różnicowego czy skid-steer, napęd Ackermana nie wymusza poślizgu — każde koło podąża za odpowiednią krzywizną podczas skrętu.

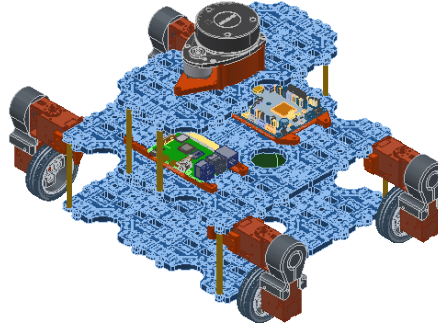
Sterowanie takim układem wymaga precyzyjnego ustawienia kątów przednich kół tak, aby punkt przecięcia ich osi pokrywał się z punktem obrotu pojazdu.

Ograniczeniem tego napędu jest promień skrętu, zależny od maksymalnego kąta wychylenia przednich kół, co ogranicza manewrowość w ciasnych przestrzeniach w porównaniu do platform różnicowych czy skid-steer, które mogą obracać się w miejscu. Napęd Ackermana idealnie sprawdza się w zastosowaniach, gdzie wymagana jest stabilność i przewidywalność ruchu, np. w autonomicznych pojazdach lub robotach poruszających się po utwardzonych nawierzchniach.



Rys. 2.8: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.2.5 Stanowisko 4 – napęd omni (swerve drive)

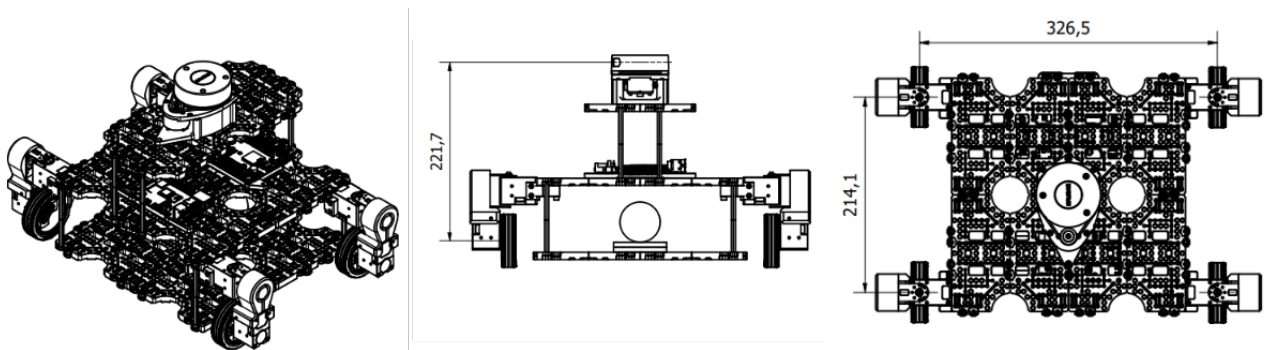


Rys. 2.9: Rzut izometryczny robota mobilnego z napędem swerve drive.

Napęd typu *swerve drive* to zaawansowany system stosowany w robotach mobilnych, zapewniający wyjątkową manewrowość i swobodę ruchu. Każde koło jest zamocowane na własnej osi obrotu i może niezależnie zmieniać zarówno kierunek, jak i prędkość. Dzięki temu robot może poruszać się płynnie w dowolnym kierunku — także bokiem lub po skosie (omnidrive) — bez zmiany swojej orientacji.

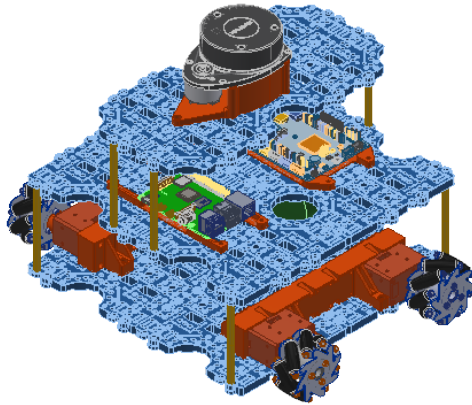
Sterowanie takim napędem wymaga precyzyjnej kontroli, ponieważ zarówno kąt skrętu, jak i prędkość każdego z kół muszą być niezależnie regulowane. Dzięki tej niezależności robot z napędem swerve może realizować złożone trajektorie, np. omijać przeszkody w sposób ciągły lub precyzyjnie dojeżdżać do punktu docelowego bez konieczności obracania całego korpusu. Jest to szczególnie przydatne w środowiskach, gdzie wymagana jest duża manewrowość i szybkie dostosowywanie kierunku ruchu, na przykład w magazynach, czy przy liniach produkcyjnych.

W napędzie swerve nie występuje wymuszony poślizg kół, dzięki czemu ruch jest płynny i dokładnie kontrolowany. Odometria robota — czyli wyznaczanie jego pozycji na podstawie ruchu kół — jest w tym przypadku znacznie dokładniejsza niż w napędzie skid-steer, gdzie poślizg powoduje błędy. Rozwiązanie to najlepiej sprawdza się tam, gdzie wymagana jest duża manewrowość i elastyczność, np. w magazynach lub przy liniach produkcyjnych.



Rys. 2.10: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.2.6 Stanowisko 5 – napęd omni (mecanum)

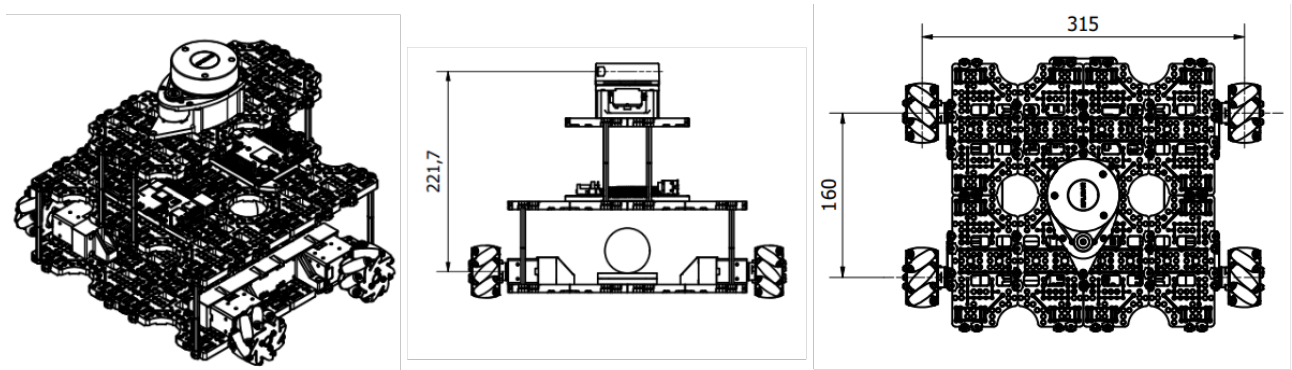


Rys. 2.11: Rzut izometryczny robota mobilnego z napędem mecanum.

Napęd typu *mecanum* zapewnia pełną swobodę ruchu robota w dowolnym kierunku dzięki zastosowaniu specjalnych kół z rolkami ustawionymi pod kątem 45° . Każde koło może obracać się niezależnie, a odpowiednie kombinacje prędkości obrotowych pozwalają uzyskać ruch do przodu, do tyłu, na boki lub po przekątnej, bez konieczności zmiany orientacji robota.

Sterowanie napędem mecanum polega na precyzyjnym regulowaniu prędkości i kierunku obrotu każdego koła. Kiedy koła obracają się w określonym układzie, prędkości wynikowe rolek dodają się, aby wywołać ruch w pożądanym kierunku. Na przykład, aby poruszać się do przodu, wszystkie koła obracają się w tym samym kierunku, natomiast ruch w bok uzyskuje się przez przeciwny obrót kół po przekątnych. Dzięki temu możliwe jest wykonywanie bardzo złożonych manewrów, takich jak przesuwanie się w bok czy obrót w miejscu.

Ograniczeniem kół mecanum jest zmniejszona efektywność na nierównym lub szorstkim terenie, gdzie rolki mogą tracić kontakt z podłożem, co prowadzi do utraty trakcji i uniemożliwi jazdę. Mimo to napęd mecanum doskonale sprawdza się na płaskich i gładkich powierzchniach np. w systemach magazynowych lub robotach serwisowych.

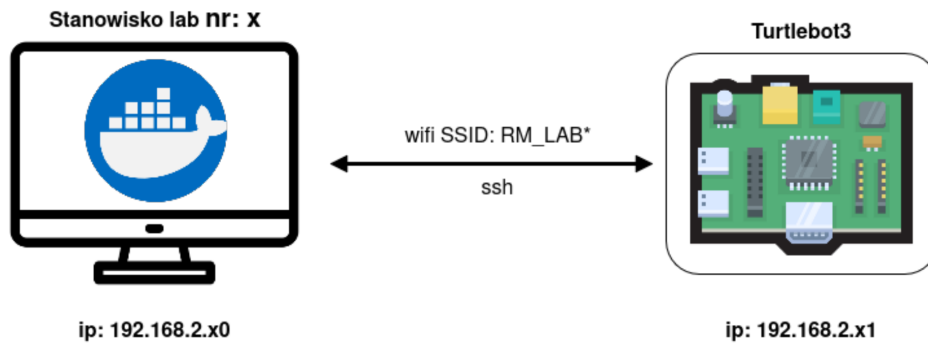


Rys. 2.12: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.3 Konfiguracja adresów IP i ROS_DOMAIN_ID

Aby połączyć się z komputera z robotem, należy znać numer stanowiska, adres IP robota i komputera stanowiskowego oraz wartość zmiennej **ROS_DOMAIN_ID**.

Połączenie z robotem oraz uruchomienie odpowiednich węzłów ROS 2 możliwe jest po wcześniejszym zalogowaniu się na robota z użyciem terminala przez **ssh**.

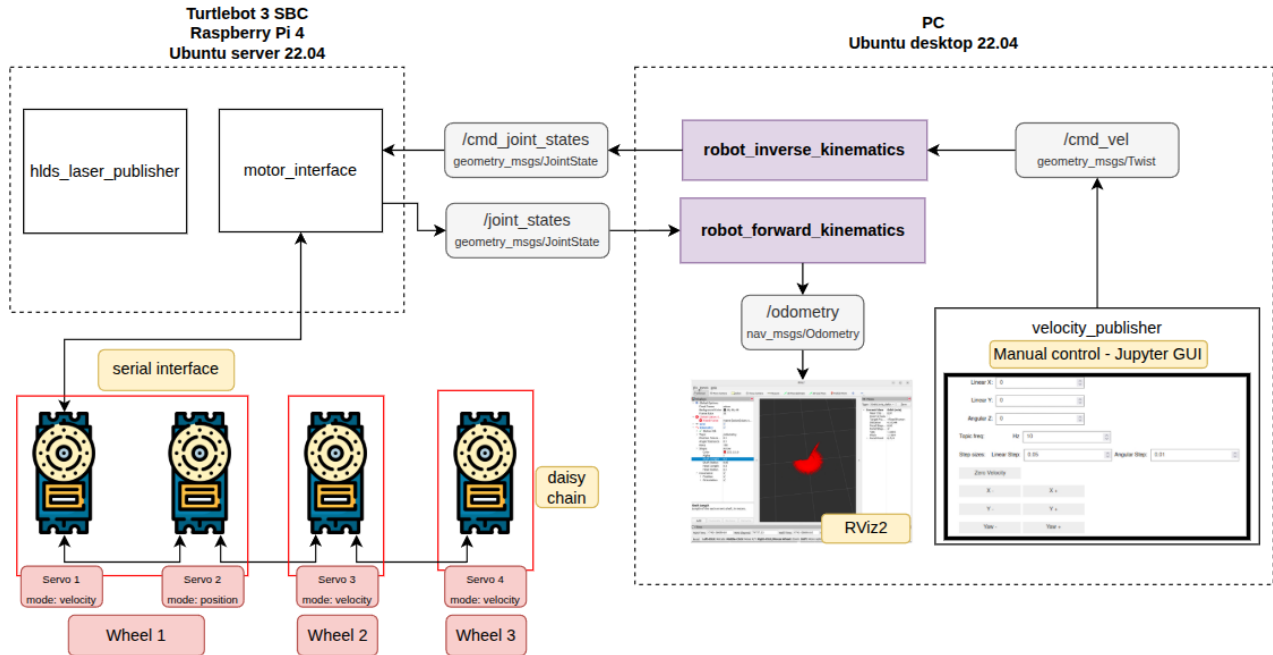


Rys. 2.13: Ogólny schemat komunikacji pojedynczego stanowiska z robotem.

Urządzenie	IP	ROS_DOMAIN_ID
Główny router Teltonika RUTM10	192.168.2.1	
PC stanowisko 1	192.168.2.10	10
PC stanowisko 2	192.168.2.20	20
PC stanowisko 3	192.168.2.30	30
PC stanowisko 4	192.168.2.40	40
PC stanowisko 5	192.168.2.50	50
TB3_1 RPI4 (2 wheel diff)	192.168.2.11	10
TB3_2 RPI4 (skid-steer 4 wheels)	192.168.2.21	20
TB3_3 RPI4 (car like 4 wheels)	192.168.2.31	30
TB3_4 RPI4 (swerve drive 4 wheels)	192.168.2.41	40
TB3_5 RPI4 (mecanum 4 wheels)	192.168.2.51	50
TB4_1 RPI4 (Livox 360)	192.168.2.12	10
TB4_2 RPI4 (OAK camera)	192.168.2.22	20
TB4_3 RPI4	192.168.2.32	30
TB4_4 RPI4	192.168.2.42	40
TB4_5 RPI4	192.168.2.52	50

Rys. 2.14: Konfiguracja sieci w laboratorium.

2.4 Komunikacja pomiędzy węzłami ROS2



Rys. 2.15: Architektura komunikacji w ROS 2.

2.4.1 Robot

Na robocie uruchamiane są następujące węzły:

- `motor_interface` — odpowiada za:
 - odbieranie na temacie `/cmd_joint_states` zadanych prędkości oraz kątów skrętu dla każdego z kół, a następnie przesyłanie tych danych bezpośrednio do silników robota,
 - odbieranie informacji zwrotnej z enkoderów w napędach o aktualnym stanie — prędkości i kątach skrętu każdego z kół — oraz publikowanie tych danych na temacie `/joint_states`.
- `hlds_laser_publisher` — obsługuje skaner laserowy.

Napędy w robocie połączone są ze sobą **szeregowo**. Mogą pracować w trybie **pozycyjnym** lub **prędkościowym**. Na Rys. 2.15 przedstawiono jedynie przykładowe napędy — w zależności od kinematyki, jedno koło może być napędzane przez jedno lub dwa serwa.

W przypadku, gdy na dane koło wysyłane są wyłącznie prędkości sterujące (np. w kinematyce z napędem różnicowym), jedno koło odpowiada jednemu napędowi. Liczba kół jest więc równa liczbie napędów.

Inaczej jest w kinematyce typu *omni* (swerve drive), w której każde koło składa się z dwóch serw: jedno pracuje w trybie prędkościowym (odpowiada za jazdę), a drugie w trybie pozycyjnym (umożliwia skręcanie).

2.4.2 Komputer PC

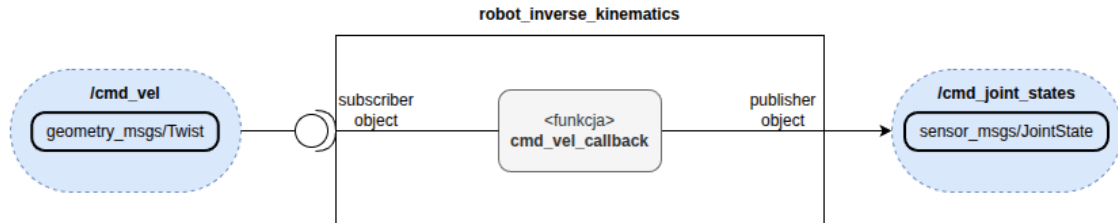
Po stronie komputera PC uruchamiane są następujące węzły:

- `robot_inverse_kinematics` — realizacja kinematyki odwrotnej (jedno z ćwiczeń do wykonania),
- `robot_forward_kinematics` — realizacja kinematyki prostej (jedno z ćwiczeń do wykonania),

- `velocity_publisher` — interfejs do sterowania robotem; publikuje prędkości sterujące robota na temacie `/cmd_vel`.

Węzeł `robot_inverse_kinematics`

Kinematyka odwrotna polega na przeliczeniu prędkości sterujących robotem (zadanej prędkości postępowej i obrotowej w układzie robota) zgodnie z jego modelem kinematycznym na prędkości oraz kąty skrętu poszczególnych kół.



Rys. 2.16: Węzeł realizujący kinematykę odwrotną (`robot_inverse_kinematics`).

Węzeł odbiera z tematu `/cmd_vel` (Rys. 2.17) informacje o zadanych prędkościach sterujących wysyłanych przez operatora (np. z interfejsu graficznego), przetwarza je na prędkości i kąty skrętu kół (jeśli wymaga tego dana kinematyka), a następnie publikuje je na temacie `/cmd_joint_states` (Rys. 2.18).

```
ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
  float64 x
  float64 y
  float64 z
Vector3  angular
  float64 x
  float64 y
  float64 z
ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$
```

Rys. 2.17: Format wiadomości wysyłanej na temacie `/cmd_vel`.

```
ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$ ros2 interface show sensor_msgs/msg/JointState
# This is a message that holds data to describe the state of a set of torque controlled joints
#
# The state of each joint (revolute or prismatic) is defined by:
# * the position of the joint (rad or m),
# * the velocity of the joint (rad/s or m/s) and
# * the effort that is applied in the joint (Nm or N).
#
# Each joint is uniquely identified by its name
# The header specifies the time at which the joint states were recorded. All the joint states
# in one message have to be recorded at the same time.
#
# This message consists of a multiple arrays, one for each part of the joint state.
# The goal is to make each of the fields optional. When e.g. your joints have no
# effort associated with them, you can leave the effort array empty.
#
# All arrays in this message should have the same size, or be empty.
# This is the only way to uniquely associate the joint name with the correct
# states.

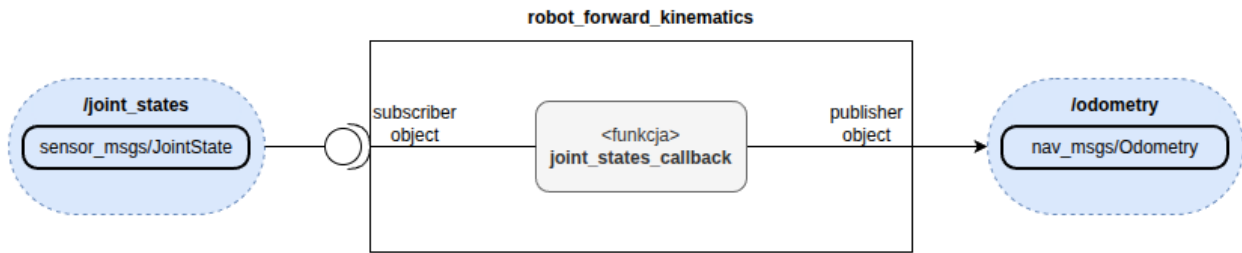
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id

string[] name
float64[] position
float64[] velocity
float64[] effort
ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$
```

Rys. 2.18: Format wiadomości wysyłanej na tematach `/cmd_joint_states` i `/joint_states`.

Węzeł robot_forward_kinematics

Węzeł odpowiada za realizację kinematyki prostej oraz wyznaczenie odometrii. Kinematyka prosta polega na przeliczeniu prędkości i kątów skrętu (jeśli występują) poszczególnych kół na wypadkową prędkość robota.



Rys. 2.19: Węzeł realizujący kinematykę prostą (robot_forward_kinematics).

Węzeł odbiera z tematu /joint_states (Rys. 2.18) informacje o aktualnych prędkościach i kątach skrętu kół. Na podstawie tych danych oblicza rzeczywistą prędkość robota (a nie kół), wyznacza odometrię i publikuje ją na temacie /odometry (Rys. 2.20).

```

ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$ ros2 interface show nav_msgs/msg/Odometry
# This represents an estimate of a position and velocity in free space.
# The pose in this message should be specified in the coordinate frame given by header.frame_id
# The twist in this message should be specified in the coordinate frame given by the child_frame_id

# Includes the frame id of the pose parent.
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
    string frame_id

# Frame id the pose points to. The twist is in this coordinate frame.
string child_frame_id

# Estimated pose that is typically relative to a fixed world frame.
geometry_msgs/PoseWithCovariance pose
  Pose pose
    Point position
      float64 x
      float64 y
      float64 z
    Quaternion orientation
      float64 x 0
      float64 y 0
      float64 z 0
      float64 w 1
    float64[36] covariance

# Estimated linear and angular velocity relative to child_frame_id.
geometry_msgs/TwistWithCovariance twist
  Twist twist
    Vector3 linear
      float64 x
      float64 y
      float64 z
    Vector3 angular
      float64 x
      float64 y
      float64 z
    float64[36] covariance
ubuntu@agnieszka-ThinkPad-P16-Gen-2:/$
  
```

Rys. 2.20: Format wiadomości wysyłanej na temacie /odometry.